Lab 4: Stacks & Queues

Topics for today:

- Infix -> Postfix evaluation
- Circular array queue
- Two-stack queue

Icebreaker!

Introduce yourself through:

- Name
- Major
- Why you're taking this class
- What pet would you have

Agenda & learning goals

Agenda

- Stacks refresher + trace practice
- Evaluate postfix expressions using a stack
- Queue refresher
- Circular array queue
- Queue implemented using two stacks

By the end, students can…

- Explain LIFO vs FIFO and core operations
- Evaluate a postfix expression step-by-step
- Trace enqueue/dequeue in a circular array queue
- Trace and explain a two-stack queue (amortized O(1))

Keep students doing quick traces—most errors are “pointer drift” or operand order.

Stacks

Stacks refresher

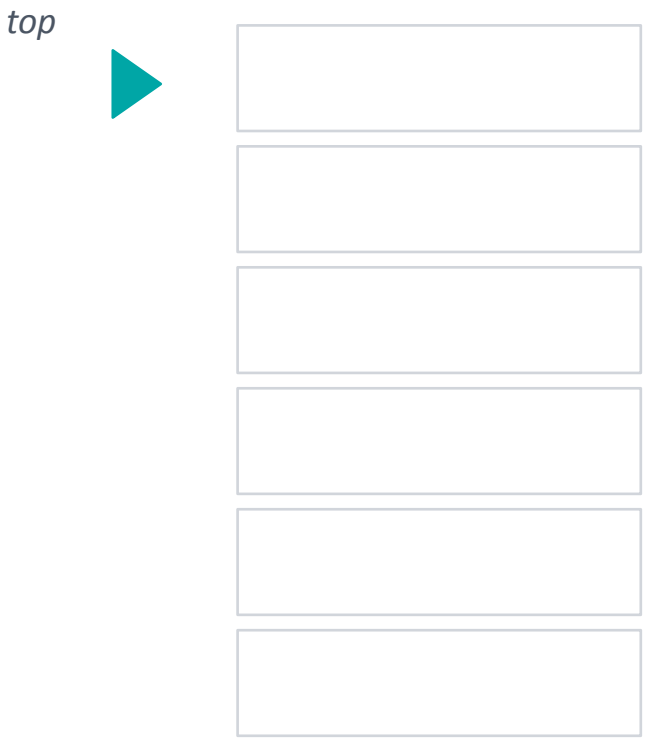
Core idea: LIFO (last in, first out)

- Only the “top” is accessible
- push(): add to top
- pop(): remove + return top
    - also known as top()
- peek(): read top (no removal)

Implementing stacks with python lists:

- push() → append()
- pop() → pop()
- peek() → li[-1]

Mental model



push / pop happen here

Warm-up: trace the stack

Starting from an empty stack, what happens?

Operations:

```
push(3), push(7), pop(), push(2), peek()
```

Questions:

- What value is popped?
- What value is returned by peek()?

Using a stack to evaluate mathematical equations: Step 1

Goal: Evaluate infix expressions such as (2 + 3) * 24 - 15 using stacks
Step 1: Convert infix expression to postfix expression
Step 2: Use stacks to evaluate postfix expression

INFIX TO POSTFIX ALGORITHM

```
for each character ch in the infix expression, do
   if ch is an operand (integer), then
      append the operand to postfixExpression
   else if ch = opening parenthesis (, then
      push ( onto the stack
   else if ch = closing parenthesis ), then
      while stack is not empty and stack top ≠ (, do
         pop and add item from stack to postfixExpression
      done
      pop ( from the stack
   else if ch is a unary operator, then
      push ch onto the stack
   else if ch is a binary operator, then
      while (stack is not empty AND precedence of ch <= precedence of stack top element), do
         pop and add item from stack into postfixExpression
      done
      push ch onto the stack
done
while the stack is not empty, do
   pop and add item from the stack to postfixExpression
done
return postfix
```

Using a stack to evaluate mathematical equations: Step 2

Goal: Evaluate infix expressions such as (2 + 3) * 24 - 15 using stacks
Step 1: Convert infix expression to postfix expression
Step 2: Use stacks to evaluate postfix expression

Evaluate Postfix Algorithm

```
for each character ch in the postfix expression, do
   if ch is an operand (integer), then
      push ch onto the stack
   else if ch is a unary operator ~, then
      pop value from the stack, negate it, and push the negated value back onto the stack
   else if ch is binary operator, then
      pop two operands off the stack
      evaluate them and push the integer evaluation back onto the stack
done
pop the final value off the stack and return it
```

Visual: Infix → Postfix

Infix : (2 * 3) + 2

MyStack:

Postfix:

Visual: Infix → Postfix

Infix : (2 * 3) + 2

Open parentheses: push onto stack

MyStack: (

Postfix:

Visual: Infix → Postfix

Infix : (**2** * 3) + 2

Operand: append the operand to postfixExpression

MyStack: (

Postfix: 2

Visual: Infix → Postfix

Infix : (2 * 3) + 2

```
Binary Operator:
while (stack is not empty AND precedence of ch <= precedence of stack top element), do
    pop and add item from stack into postfixExpression
done
push ch onto the stack
```

MyStack: (*

Postfix: 2

Visual: Infix → Postfix

Infix : (2 * 3) + 2

Operand: append the operand to postfixExpression

MyStack:	(	*
Postfix:	2	3

Visual: Infix → Postfix

Infix : (2 * 3) + 2

```
closing parenthesis:
while stack is not empty and stack top ≠ (, do
    pop and add item from stack to postfixExpression
done
pop ( from the stack
```

MyStack:

Postfix: 2 3 *

Visual: Infix → Postfix

Infix : (2 * 3) + 2

```
Binary Operator:
while (stack is not empty AND precedence of ch <= precedence of stack top element), do
    pop and add item from stack into postfixExpression
done
push ch onto the stack
```

MyStack: +

Postfix: 2 3 *

Visual: Infix → Postfix

Infix : (2 * 3) + 2

Operand: append the operand to postfixExpression

MyStack: +

Postfix: 2 3 * 2

Visual: Infix → Postfix

Infix : (2 * 3) + 2

```
while the stack is not empty, do
    pop and add item from the stack to postfixExpression
done
return postfix
```

MyStack:

Postfix: 2 3 * 2 +

Visual: Postfix → Evaluation

Postfix: 2 3 * 2 +

MyStack:

Visual: Postfix → Evaluation

Postfix: **2** 3 * 2 +

Operand: push ch onto the stack

MyStack: 2

Visual: Postfix → Evaluation

Postfix: 2 3 * 2 +

Operand: push ch onto the stack

MyStack: 2 3

Visual: Postfix → Evaluation

Postfix: 2 3 * 2 +

binary operator:
Pop two operands off the stack
Evaluate them and push the integer evaluation back onto the stack

MyStack: 6

Visual: Postfix → Evaluation

Postfix: 2 3 * **2** +

Operand: push ch onto the stack

MyStack: 6 2

Visual: Postfix → Evaluation

Postfix: 2 3 * 2 +

binary operator:
Pop two operands off the stack
Evaluate them and push the integer evaluation back onto the stack

MyStack: 8

Visual: Postfix → Evaluation

Postfix: 2 3 * 2 +

pop the final value off the stack and return it

MyStack:

(2 * 3) + 2 = **8**

Partner practice (postfix)

Work in pairs. Evaluate using a stack:

1) `2 3 + 5 *`

Write out the stack at each step

2) `10 2 8 * + 3 -`

Write out the stack at each step

Quick debrief: Can a few pairs narrate the pops for each operator?

Queue Time!

Queues refresher

Core idea: FIFO

What does FIFO stand for and what does it mean?

What are some real-world applications of queues?

Queue Functionalities

- q.append(x)
  - add x to the end
- q.popleft()
  - remove + return from the front
- q[0]
  - peak the front element without removing
- len(q)
  - size of the queue
- if q
  - if the queue is non-empty

Queues refresher

Queue Functionalities

- q.append(x) → add x to the end
- q.popleft() → remove + return from the front
- q[0] → peek the front element without removing it
- len(q) → size of the queue
- if q → if the queue is non-empty

Trace through queue

Starting empty:

```
q = deque()
q.append(A)
q.append(B)
q.popleft()
q.append(C)
q.popleft()
```

1. What order are elements removed?
2. What remains in the queue at the end?

Answer:

1. *A, B*
2. *C*

Queues refresher

Trace through queue

```
q = deque()
q.append(C)
q.append(A)
q.popleft()
q.append(T)
q.popleft()
q.popleft()
q.popleft()
```

Trace through this code. What error will appear? Why?

Answer: IndexError because the queue is empty

Trace through queue

```
q = deque()
q.append(A)
q.append(P)
q.append(P)
q.popleft()
q.append(L)
q.popleft()
q.append(E)
if q:
    print(q[0])
print(len(q))
```

What will we print out?

Answer: P 3

Follow-up: why do we use **if q:**?

Circular array queue

Queue where front and rear indices wrap around to the beginning when the end of the array is reached

Why circular?

- Allows you to reuse freed space at start of array
- Wrap indices with modulo arithmetic
- Track size to distinguish empty vs full

Common invariants:

```
front = index of current first element
rear  = index where next enqueue writes
empty when size==0
full when size==capacity
advance: idx = (idx+1) % capacity
```

Wrap-around picture

“after index 4, next is 0”

- append: write at rear; rear=(rear+1)%cap; size++
- dequeue: read at front; front=(front+1)%cap; size–

Normal queue can “run out of space” even when there are empty slots at front

Circular queue operations

append(x)

- If size == cap: FULL (reject)
- arr[rear] = x
- rear = (rear + 1) % cap
- size += 1

popleft()

- If size == 0: EMPTY (reject)
- x = arr[front]
- front = (front + 1) % cap
- size -= 1
- return x

Why track size?

- Without size, front==rear is ambiguous
- size==0 ⇒ empty
- size==cap ⇒ full

Teaching move: have students say “write, advance, size++” out loud.

Circular array queue: Python implementation

A minimal implementation (rear = next-write):

```
class CircularQueue:
    def __init__(self, cap):
        self.arr = [None] * cap
        self.cap = cap
        self.front = 0
        self.rear = 0  # next write
        self.size = 0

    def is_empty(self):
        return self.size == 0

    def is_full(self):
        return self.size == self.cap

    def append(self, x):
        if self.is_full():
            raise IndexError('full')
        self.arr[self.rear] = x
        self.rear = (self.rear + 1) % self.cap
        self.size += 1

    def popleft(self):
        if self.is_empty():
            raise IndexError('empty')
        x = self.arr[self.front]
        self.arr[self.front] = None
        self.front = (self.front + 1) % self.cap
        self.size -= 1
        return x

    def peek(self):
        if self.is_empty():
            raise IndexError('empty')
        return self.arr[self.front]

    def __len__(self):
        return self.size
```

Note: clearing arr[front] is optional; it helps visualize/debug.

Activity: trace a circular queue

Capacity = 5. Start: front=0, rear=0, size=0

Operations:

- q.append(10)
- q.append(20)
- q.append(30)
- q.popleft()
- q.popleft()
- q.append(40)
- q.append(50)
- q.append(60)
- q.append(70)

Step	front	rear	size
1			
2			
3			
4			
5			
6			
7			
8			
9			

Array slots (indices 0..4):

0	1	2	3	4

Questions

- When do indices wrap?
- What does it mean when front == rear?
- What happens if we have q.append(80) next?

- *Advance past last index, then idx = (idx+1)%5*
- *queue is empty or queue is full (after wrapping)*
- *queue is full, so it should fail*

Solution: circular queue trace

Capacity=5, start (front=0, rear=0, size=0). Using rear=next-write.

Step	Op	front	rear	size
1	append(10)	0	1	1
2	append(20)	0	2	2
3	append(30)	0	3	3
4	popleft()	1	3	2
5	popleft()	2	3	1
6	append(40)	2	4	2
7	append(50)	2	0	3
8	append(60)	2	1	4
9	append(70)	2	2	5

```
[10, _, _, _, _]
[10, 20, _, _, _]
[10, 20, 30, _, _]
[_ , 20, 30, _, _]
[_ , _ , 30, _, _]
[_ , _ , 30, 40, _]
[_ , _ , 30, 40, 50]
[60, _ , 30, 40, 50]
[60, 70, 30, 40, 50]
```

Wrap moment: step 7 rear goes 4⟶0 (mod 5). Queue becomes full at step 9.

Queue using two stacks

Idea

- inStack handles append(x) (push)
- outStack handles leftpop() (pop)
- When outStack is empty, transfer all items from inStack to outStack
- Transfer reverses order, exposing the oldest element on top

Performance: append() is O(1); popleft() is amortized O(1) (each item moves stacks at most once)

Run Time

- append(x) →O(1)
- leftpop() → amortized O(1)

Worst-case a leftpop() can take O(n) during transfer, but it's amortized (averaged over many operations), with each element transferring at most once from inStack to outStack

Two-stack queue: Python implementation

8 min

Amortized O(1): each element moves from inStack → outStack at most once.

```python
class TwoStackQueue:
    def __init__(self):
        self.inStack = []
        self.outStack = []

    def is_empty(self):
        return not self.inStack and not self.outStack

    def append(self, x):
        self.inStack.append(x)

    def _pour(self):
        # move everything only when needed
        while self.inStack:
            self.outStack.append(self.inStack.pop())

    def popleft(self):
        if not self.outStack:
            self._pour()
        if not self.outStack:
            raise IndexError('empty')
        return self.outStack.pop()

    def peek(self):
        if not self.outStack:
            self._pour()
        if not self.outStack:
            raise IndexError('empty')
        return self.outStack[-1]

    def __len__(self):
        return len(self.inStack) + len(self.outStack)
```

Note: lists are stacks here (append/pop at end are O(1) amortized).

Trace

Trace this sequence with 2 stacks

```
append(1), append(2), append(3), popleft(), append(4), popleft(), popleft()
```

append(1):
- inStack = [1], outStack = []

append(2):
- inStack = [1, 2], outStack = []

append(3):
- inStack = [1, 2, 3], outStack = []

popleft(): → returns [1]
- outStack is empty, so transfer all inStack to outStack
  - inStack = [], outStack = [3, 2, 1]
- inStack = [], outStack = [3, 2]

append(4):
- inStack = [4], outStack = [3, 2]

popleft(): → returns 2
- inStack = [4], outStack = [3]

popleft(): → returns 3
- inStack = [4], outStack = []

Questions

What values are dequeued (in order)?
When does the pour happen?
What remains at the end?

Answer

Dequeued: 1, 2, 3
Pour happens on the first dequeue
Remaining: 4

Recap: Circular queue uses modulo; two-stack queue uses occasional pours.

Knowledge Check

Closing Questions

- Evaluate postfix: 3 4 2 * 1 5 - / +
  - 1
- Circular queue (cap=4): if front=3, rear=1, size=2. After enqueue, what is rear?
  - rear = (rear+1)%4 = 2
- Two-stack queue: when do we transfer from inStack → outStack?
  - when outStack is empty

Recap: stack = LIFO; queue = FIFO; postfix uses stack pops; circular queue uses modulo + size; two-stack queue is amortized O(1).